Module Guide for TeLaC

Team 24, Maca Boca
Javayria Mudassar
Yilin Jiang
Nour Fawaz
Catherine Yang

November 13, 2026

1 Revision History

Date	Version	Notes
Date 1	1.0	Notes
Date 2	1.1	Notes

2 Reference Material

This section records information for easy reference.

2.1 Abbreviations and Acronyms

symbol	description
AC	Anticipated Change
DAG	Directed Acyclic Graph
M	Module
MG	Module Guide
OS	Operating System
R	Requirement
SC	Scientific Computing
SRS	Software Requirements Specification
TeLaC	Telehealth Language Assessment for Children
SAFA	Speech Annotation For Application, Access, Accuracy
MERLS	-
UC	Unlikely Change
[etc. —SS]	[... —SS]

Contents

1 Revision History **i**

2 Reference Material **ii**
2.1 Abbreviations and Acronyms ii

3 Introduction **1**

4 Anticipated and Unlikely Changes **2**
4.1 Anticipated Changes 2
4.2 Unlikely Changes 2

5 Module Hierarchy **2**

6 Connection Between Requirements and Design **4**

7 Module Decomposition **4**
7.1 Hardware Hiding Modules (M??) 4
7.2 Behaviour-Hiding Module 4
7.2.1 Input Format Module (M??) 5
7.2.2 Video Recording Module (M6) 5
7.2.3 Transcription Service Interface Module (M1) 5
7.3 Software Decision Module 5
7.3.1 ML Transcription Model Module (M11) 6

8 Traceability Matrix **6**

9 Use Hierarchy Between Modules **7**

10 User Interfaces **8**

11 Design of Communication Protocols **8**

12 Database Design **8**

13 Timeline **8**
13.1 Module Implementation Timeline (Starting Jan 4) 8
13.1.1 User Interface (UI) Modules 8
13.1.2 Backend Modules 9

14 Generative AI Use Disclosure **10**
14.1 AI Tools Used 10
14.2 How and Where Used 10
14.3 Verification of AI-Generated Content 11

List of Tables

1 Module Hierarchy 3
2 Trace Between Requirements and Modules 6
3 Trace Between Anticipated Changes and Modules 7

List of Figures

1 Use hierarchy among modules 7

3 Introduction

Decomposing a system into modules is a commonly accepted approach to developing software. A module is a work assignment for a programmer or programming team (**?**). We advocate a decomposition based on the principle of information hiding (**?**). This principle supports design for change, because the "secrets" that each module hides represent likely future changes. Design for change is valuable in SC, where modifications are frequent, especially during initial development as the solution space is explored.

Our design follows the rules layed out by **?**, as follows:

- System details that are likely to change independently should be the secrets of separate modules.
- Each data structure is implemented in only one module.
- Any other program that requires information stored in a module's data structures must obtain it by calling access programs belonging to that module.

After completing the first stage of the design, the Software Requirements Specification (SRS), the Module Guide (MG) is developed (**?**). The MG specifies the modular structure of the system and is intended to allow both designers and maintainers to easily identify the parts of the software. The potential readers of this document are as follows:

- New project members: This document can be a guide for a new project member to easily understand the overall structure and quickly find the relevant modules they are searching for.
- Maintainers: The hierarchical structure of the module guide improves the maintainers' understanding when they need to make changes to the system. It is important for a maintainer to update the relevant sections of the document after changes have been made.
- Designers: Once the module guide has been written, it can be used to check for consistency, feasibility, and flexibility. Designers can verify the system in various ways, such as consistency among modules, feasibility of the decomposition, and flexibility of the design.

The rest of the document is organized as follows. Section 4 lists the anticipated and unlikely changes of the software requirements. Section 5 summarizes the module decomposition that was constructed according to the likely changes. Section 6 specifies the connections between the software requirements and the modules. Section 7 gives a detailed description of the modules. Section 8 includes two traceability matrices. One checks the completeness of the design against the requirements provided in the SRS. The other shows the relation between anticipated changes and the modules. Section 9 describes the use relation between modules.

4 Anticipated and Unlikely Changes

This section lists possible changes to the system. According to the likeliness of the change, the possible changes are classified into two categories. Anticipated changes are listed in Section 4.1, and unlikely changes are listed in Section 4.2.

4.1 Anticipated Changes

Anticipated changes are the source of the information that is to be hidden inside the modules. Ideally, changing one of the anticipated changes will only require changing the one module that hides the associated decision. The approach adapted here is called design for change.

AC1: The specific hardware on which the software is running.

AC2: The format of the initial input data.

...

[Anticipated changes relate to changes that would be made in requirements, design or implementation choices. They are not related to changes that are made at run-time, like the values of parameters. —SS]

4.2 Unlikely Changes

The module design should be as general as possible. However, a general system is more complex. Sometimes this complexity is not necessary. Fixing some design decisions at the system architecture stage can simplify the software design. If these decision should later need to be changed, then many parts of the design will potentially need to be modified. Hence, it is not intended that these decisions will be changed.

UC1: Input/Output devices (Input: File and/or Keyboard, Output: File, Memory, and/or Screen).

...

5 Module Hierarchy

This section provides an overview of the module design. Modules are summarized in a hierarchy decomposed by secrets in Table 1. The modules listed below, which are leaves in the hierarchy tree, are the modules that will actually be implemented.

User Interface Modules

M1: Transcription Service Interface Module

M2: Frontend Module

Backend Modules

M3: Device I/O Module

M4: API Gateway Module

M5: Account System Module

M6: Video Recording Module

M7: Report Generation Module

M8: Assessment Review Module

M9: Database Module

M10: Encryption Module

M11: ML Transcription Module

M12: MERLS Module

Level 1	**Level 2**
Hardware-Hiding	Device I/O Module
Behaviour-Hiding	Frontend Module Account System Module Video Recording Module Transcription Service Interface Module Report Generation Module Assessment Review Module MERLS Module
Software Decision	API Gateway Module Database Module Encryption Module ML Transcription Model Module

Table 1: Module Hierarchy

6 Connection Between Requirements and Design

The design of the system is intended to satisfy the requirements developed in the SRS. In this stage, the system is decomposed into modules. The connection between requirements and modules is listed in Table 2.

[The intention of this section is to document decisions that are made "between" the requirements and the design. To satisfy some requirements, design decisions need to be made. Rather than make these decisions implicit, they are explicitly recorded here. For instance, if a program has security requirements, a specific design decision may be made to satisfy those requirements with a password. —SS]

7 Module Decomposition

Modules are decomposed according to the principle of "information hiding" proposed by **?**. The *Secrets* field in a module decomposition is a brief statement of the design decision hidden by the module. The *Services* field specifies *what* the module will do without documenting *how* to do it. For each module, a suggestion for the implementing software is given under the *Implemented By* title. If the entry is *OS*, this means that the module is provided by the operating system or by standard programming language libraries. *TeLaC* means the module will be implemented by the TeLaC software.

Only the leaf modules in the hierarchy have to be implemented. If a dash (–) is shown, this means that the module is not a leaf and will not have to be implemented.

7.1 Hardware Hiding Modules (M??)

Secrets: The data structure and algorithm used to implement the virtual hardware.

Services: Serves as a virtual hardware used by the rest of the system. This module provides the interface between the hardware and the software. So, the system can use it to display outputs or to accept inputs.

Implemented By: OS

7.2 Behaviour-Hiding Module

Secrets: The contents of the required behaviours.

Services: Includes programs that provide externally visible behaviour of the system as specified in the software requirements specification (SRS) documents. This module serves as a communication layer between the hardware-hiding module and the software decision module. The programs in this module will need to change if there are changes in the SRS.

Implemented By: –

7.2.1 Input Format Module (M??)

Secrets: The format and structure of the input data.

Services: Converts the input data into the data structure used by the input parameters module.

Implemented By: [Your Program Name Here]

Type of Module: [Record, Library, Abstract Object, or Abstract Data Type] [Information to include for leaf modules in the decomposition by secrets tree.]

7.2.2 Video Recording Module (M6)

Secrets: How the recording quality is judged, how a recording moves from being captured to saved, and how failed uploads are retried and verified.

Services: Checks that the audio and video are good enough before an assessment, records the session, and saves the recording so it can be played back and backed up.

Implemented By: TeLaC

Type of Module: Abstract Object

7.2.3 Transcription Service Interface Module (M1)

Secrets: The format of a transcript, the rules for editing, labelling and verifying transcript segments, and the format of exported annotations.

Services: Generates a draft transcript for an assessment recording, and lets the clinician correct the text, assign speaker roles, verify segments, mark the transcript complete and export the annotations.

Type of Module: Library

7.3 Software Decision Module

Secrets: The design decision based on mathematical theorems, physical facts, or programming considerations. The secrets of this module are *not* described in the SRS.

Services: Includes data structure and algorithms used in the system that do not provide direct interaction with the user.

Implemented By: –

7.3.1 ML Transcription Model Module (M11)

Secrets: The speech recognition model used, how audio is split into chunks, and how the model's confidence and the audio level are turned into flags.

Services: Converts an audio file into timed text segments, each with a confidence score, a detected language and a flag showing whether it needs review.

Implemented By: TeLaC (using OpenAI's Whisper speech-to-text model)

Type of Module: Library

8 Traceability Matrix

This section shows two traceability matrices: between the modules and the requirements and between the modules and the anticipated changes.

Req.	Modules
FR1	M1, M11
FR2	M1
FR3	M1
FR6	M1
FR15	M1
FR19	M1
NFR7	M6, M1, M11
NFR8	M1
NFR9	M6
NFR10	M6, M11
NFR13	M1
NFR14	M1
NFR17	M1, M11
NFR18	M1
NFR21	M1
HA-VR1	M6
HA-VR2	M6
HAMLTM1	M11
HAMLTM2	M11

Table 2: Trace Between Requirements and Modules

AC	Modules
AC1	M??
AC2	M??
AC??	M??
AC??	M??
AC??	M??
AC??	M??
AC??	M??
AC??	M??
AC??	M??
AC??	M??
AC??	M??
AC??	M??

Table 3: Trace Between Anticipated Changes and Modules

9 Use Hierarchy Between Modules

In this section, the uses hierarchy between modules is provided. **?** said of two programs A and B that A *uses* B if correct execution of B may be necessary for A to complete the task described in its specification. That is, A *uses* B if there exist situations in which the correct functioning of A depends upon the availability of a correct implementation of B. Figure 1 illustrates the use relation between the modules. It can be seen that the graph is a directed acyclic graph (DAG). Each level of the hierarchy offers a testable and usable subset of the system, and modules in the higher level of the hierarchy are essentially simpler because they use modules from the lower levels.

[The uses relation is not a data flow diagram. In the code there will often be an import statement in module A when it directly uses module B. Module B provides the services that module A needs. The code for module A needs to be able to see these services (hence the import statement). Since the uses relation is transitive, there is a use relation without an import, but the arrows in the diagram typically correspond to the presence of import statement. —SS]

[If module A uses module B, the arrow is directed from A to B. —SS]

Figure 1: Use hierarchy among modules

10 User Interfaces

[Design of user interface for software and hardware. Attach an appendix if needed. Drawings, Sketches, Figma —SS]

11 Design of Communication Protocols

[If there is communication between hardware and software, it may be necessary to explain the design of the communication protocol. This section should be filled in with NA if it is not appropriate for a given project. —SS]

12 Database Design

[If a database is a significant component of the project, the database design should be summarized here. This section should be filled in with NA if it is not appropriate for a given project. —SS]

13 Timeline

13.1 Module Implementation Timeline (Starting Jan 4)

This section outlines the implementation timeline for each module. For every module, it describes the module's purpose, identifies the team member responsible for implementing it, and gives the expected start and end dates.

13.1.1 User Interface (UI) Modules

- **Jan 7 – Jan 9: Transcription Service Interface Module (1)**
  - Implement the UI for SAFA in the existing MERLS platform.
  - Add functionality to interact with the SAFA interface and transcribe.
  - **Responsibility:** Nour.
- **Jan 25 – Jan 27: Frontend Module (2)**
  - Implement the clinician frontend.
  - **Responsibility:** Catherine.

13.1.2 Backend Modules

- **Jan 4 – Jan 21: Device I/O Module (3)**
  - Add functionality to handle hardware input.
  - **Responsibility:** Yilin.
- **Jan 4 – Jan 6: Database Module (9)**
  - Implement the database that stores all assessment and patient information.
  - **Responsibility:** Javayria.
- **Jan 4 – Jan 6: ML Transcription Module (11)**
  - Integrate the existing SAFA tool into MERLS.
  - Possibly incorporate multiple views for different types of users.
  - **Responsibility:** Nour.
- **Jan 7 – Jan 9: Encryption Module (10)**
  - Implement encryption of patient information.
  - **Responsibility:** Javayria.
- **Jan 10 – Jan 12: Report Generation Module (7)**
  - Add functionality to create reports.
  - **Responsibility:** Yilin.
- **Jan 13 – Jan 15: Assessment Review Module (8)**
  - Add functionality to flag, save, delete, and mark assessment questions, including creating and deleting notes for questions.
  - Add functionality to record clinician evaluations and retrieve reviews for patient assessments.
  - **Responsibility:** Yilin.
- **Jan 16 – Jan 18: Account System Module (5)**
  - Implement user accounts.
  - Add functionality for logging in/out and authorizing clinician accounts.
  - **Responsibility:** Catherine.
- **Jan 19 – Jan 21: API Gateway Module (4)**
  - Develop an access point for modules to make their corresponding API calls.

  - **Responsibility:** Catherine.

- **Jan 22 – Jan 24: Video Recording Module (6)**
  - Implement a module to handle user recording input (audio and video) for use by the system.
  - **Responsibility:** Nour.

- **Jan 28 – Jan 31: MERLS Module (12)**
  - Connect the clinician platform to the existing MERLS patient interface.
  - **Responsibility:** Javayria.

14 Generative AI Use Disclosure

This section records all uses of generative artificial intelligence throughout the document.

14.1 AI Tools Used

- Claude Sonnet 5 (Medium)
- ChatGPT-5.6 Luna

14.2 How and Where Used

The following lists the uses of AI applied throughout the entire document:

- Fixing grammar and spelling
- Debugging possible LaTeX errors
- Performing a final check over the document at the end, comparing each section against the original document comments and the rubric to ensure nothing was missed
- Removing template comments

Section 7

- Check if secrets and services are consistent with the MIS document

Section 8

- Format both tables for LaTeX

14.3 Verification of AI-Generated Content

To check the LLM's output, each team member went through their assigned section and presented it to the group, outlining the progress they had made since the previous meeting. ALL members were able to explain their section with confidence. Prior to approving any pull request, a team member other than the section's author reviewed it to make sure it was cohesive, accurate, and in consistent with the rest of the document. Since the document did not include any factual claims, the LLM's output did not need further fact-checking.